МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
‹‹СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ››
Институт перспективной инженерии

Дисциплина «Введение в программирование и алгоритмы»

Отчет по лабораторной работе №5
«Циклы. Табуляция функций»

Выполнил студент группы
ПИЖ-б-о-26-1(2)
Киркачев М.Е.
Проверил:
Преподаватель по дисциплине
Николаев Е. И.

г. Ставрополь, 2026 г.

Цель работы: научиться использовать циклы for, while, do-while для организации повторяющихся вычислений. Освоить табуляцию функций — вывод таблицы значений функции на заданном интервале с заданным шагом. Получить навыки накопления суммы (аккумулятор), поиска минимального и максимального значения функции, организации вложенных циклов.

ХОД РАБОТЫ

https://github.com/Max2311008/my-projects/tree/main/work%20in%20C++/lab_5

ВАРИАНТ №9

Задание 1(базовый уровень)

Код:

```
while (true) {
    std::cout << "Введите конец интервала: ";
    if (std::cin >> last) {
        break;
    }
    // ввод числа с повтором ввода данных при ошибке
    std::cout << "ОШИБКА! Введите число заново.\n"; // ввод текста на экран
    std::cin.clear(); // сброс флага ошибки
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    // очистка буфера от мусора
}
while (true) {
    std::cout << "Введите шаг интервала: ";
    if (std::cin >> shag) {
        break;
    }
    // ввод числа с повтором ввода данных при ошибке
    std::cout << "ОШИБКА! Введите число заново.\n"; // ввод текста на экран
    std::cin.clear(); // сброс флага ошибки
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    // очистка буфера от мусора
}
if (first > last && std::cin.fail()) {
    std::cout << "ОШИБКА: начало интервала не может быть больше конца!" << std::endl;
    return 1;
} // Условие для правильного построения функции
if (shag <= 0) {
    std::cout << "ОШИБКА: шаг не может быть отризательным!" << std::endl;
    return 1;
}
// Условие для правильного построения функции
std::cout << "\n=====================================" << std::endl;
std::cout << " x   |  y = (x-2)**2 " << std::endl;
std::cout << "=====================================" << std::endl;
std::cout << std::fixed << std::setprecision(4);
for (double x = first; x <= last + 1e-9; x += shag) { // Использование + 1e-9 для учёта погрешности double
    double y = sqrt((x - 2)); // Вычисляем значение функции
    std::cout << std::setw(8) << x << "|" << std::setw(12) << y << std::endl;
}
std::cout << "=====================================" << std::endl;
return 0;
```

Результат

```
Консоль отладки Microsoft Visual Studio
Введите начало интервала: 13
Введите конец интервала: 25
Введите шаг интервала: 2

=====================================
 x   |   y = (x-2)**2
=====================================
 13.0000|      3.3166
 15.0000|      3.6056
 17.0000|      3.8730
 19.0000|      4.1231
 21.0000|      4.3589
 23.0000|      4.5826
 25.0000|      4.7958
=====================================
```

Задание 2(средний уровень)

Код:

```cpp
// Условие для правильного построения функции
double xFirst = first;
double yFirst = 1 / (xFirst * xFirst + 1);
double maxY = yFirst;
double maxX = xFirst;
// Инициализация минимума и максимума первым значением
std::cout << "\n$===================================$" << std::endl;
std::cout << " x   |   y = 1/(x**2 + 1) " << std::endl;
std::cout << "$===================================$" << std::endl;
std::cout << std::fixed << std::setprecision(4);
for (double x = first; x <= last + 1e-9; x += shag) { // Использование + 1e-9 для учёта погрешности double
    double y = 1 / (x * x + 1); // Вычисляем значение функции
    std::cout << std::setw(7) << x << "|" << std::setw(9) << y << std::endl;

    if (y > maxY) {
        maxY = y;
        maxX = x;
    } // Обновление максимума
}
std::cout << "$===================================$" << std::endl;
std::cout << "\nРезультаты поиска:" << std::endl;
std::cout << "Наибольшее значение: "  << maxY << std::endl;
return 0;
```

Результат

```
Консоль отладки Microsoft Visual Studio
Введите начало интервала: 15
Введите конец интервала: 120
Введите шаг интервала: 12

$===================================$
 x   |   y = 1/(x**2 + 1)
$===================================$
15.0000|    0.0044
27.0000|    0.0014
39.0000|    0.0007
51.0000|    0.0004
63.0000|    0.0003
75.0000|    0.0002
87.0000|    0.0001
99.0000|    0.0001
111.0000|    0.0001
$===================================$

Результаты поиска:
Наибольшее значение: 0.0044
```

Задание 3(повышенный уровень)

Код:

```
if (first > last || shag <= 0) {
    cout << "Ошибка ввода: начало > конца или шаг <= 0!\n";
    return 1;
}

double sum = 0.0; // Аккумулятор для интеграла

std::cout << std::fixed << std::setprecision(4);

// Цикл табуляции
for (double x = first; x <= last + 1e-9; x += shag) {
    double y = x * x;

    std::cout << std::setw(8) << x << " | " << std::setw(12) << y << std::endl;

    // Метод трапеций: крайние точки делим на 2
    if (x == first || x + shag > last + 1e-9) {
        sum += y / 2.0;
    }
    else {
        sum += y;
    }
}

sum *= shag; // Умножаем сумму на шаг

//  первообразная x^3 / 3 для нахождения путём аналитического значения
double otv = (last * last * last - first * first * first) / 3.0;

cout << "\n--- Результаты ---\n";
cout << "Метод трапеций:   " << sum << "\n";
cout << "Аналитический:  " << otv << "\n";
cout << "Погрешность:      " << abs(sum - otv) << "\n";

return 0;
```

Результат

```
Консоль отладки Microsoft Visua
Введите начало интервала: 12
Введите конец интервала: 38
Введите шаг интервала: 5
 12.0000 |     144.0000
 17.0000 |     289.0000
 22.0000 |     484.0000
 27.0000 |     729.0000
 32.0000 |    1024.0000
 37.0000 |    1369.0000

--- Результаты ---
Метод трапеций:  16412.5000
Аналитический: 17714.6667
Погрешность:     1302.1667
```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Раздел А. Основы циклов:

1. For, while, do-while
2. For используется, когда известно количество действий;
   While используется, когда неизвестно количество действий;
   Do-while – когда условие должно выполниться хотя бы один раз
3. Цикл станет бесконечным
4. Немедленный выход из цикла
5. Пропуск оставшейся части тела цикла и переход к следующему действию

Раздел Б. Табуляция функций:

1. Вычисление и вывод значений функции в ряде точек на заданном интервале с определённым шагом. Нужна для построения таблиц и графиков функции
2. For (double x = a; x <= b + 1e-9; x += h)
3. Из-за погрешности вычислений с типом double
4. Проверить область определения внутри цикла и пропускать точку
5. Через #include <iomanip> std::setw() –ширина столбцов

Раздел В. Накопление и поиск:

1. Переменная для накопления суммы. Сначала надо задать sum = 0, а потом в цикле прибавлять
2. Начинать нужно с 1 и потом в цикле умножать factorial += a
3. До цикла задать minValue, maxValue равными первому значению f(x). Потом в цикле сравнить y с minValue, maxValue и потом обновлять при необходимости
4. Найти сумму всех значений и разделить на количество точек

5. Суммирование от меньших к большему приводит к накоплению погрешности. Суммирование от больших к меньшим будет более точнее выводить

Раздел Г. Сложные циклы:

1. Циклы внутри другого цикла. Пример – таблица умножения
2. Break прерывает тот цикл, в котором он находится
3. While (true) {…if (условие) break;…}
4. Формула, выражающая следующий член ряда через предыдущий. Нужна для эффективного вычисления суммы ряда без использования функций возведения в степень и факториала на каждой итерации.
5. Для ряда неизвестно точное количество членов. Суммирование идет до тех пор, пока очередной член ряда не станет меньше заданной точности

Раздел Д. Анализ кода:

1. 01234
2. 5 раз
3. Нужно заменить sum = sum + 1/I на sum = sum +1.0/i
4. For (int i = 10; i > 0; i--) {
   Cout << I;}
5. Возникнет бесконечный цикл, так как double x никогда не станет ровно 1.0

Вывод: благодаря этой лабораторной работе я смог научиться использовать циклы for, while, do-while для организации повторяющихся вычислений. Освоил табуляцию функций — вывод таблицы значений функции на заданном интервале с заданным шагом. Получил навыки накопления суммы (аккумулятор), поиска минимального и максимального значения функции, организации вложенных циклов.